

Pengumpulan Tahap 1

Nama: Robani Diansyah

NPM: 23312246

1. api/services/kategori-service/src/main.ts

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from '../app.module';
import { ValidationPipe } from '@nestjs/common';

async function bootstrap(): Promise<void> {
  const app = await NestFactory.create(AppModule);

  // Prefix API
  app.setGlobalPrefix('api');

  // Validasi otomatis semua request body pakai DTO
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
      transform: true,
    }),
  );

  await app.listen(3001);
}
void bootstrap();
```

File ini adalah titik masuk utama dari kategori-service. Saat service dijalankan, fungsi bootstrap() yang pertama kali dipanggil.

Di dalamnya ada beberapa hal yang disetup:

Bagian	Fungsi
NestFactory.create(AppModule)	Membuat instance aplikasi NestJS berdasarkan AppModule
app.setGlobalPrefix('api')	Semua endpoint otomatis diawali /api/ - jadi misalnya /api/kategori
ValidationPipe	Validasi otomatis semua request yang masuk sebelum masuk ke controller
whitelist: true	Field yang tidak ada di DTO langsung dibuang, tidak diteruskan ke service
forbidNonWhitelisted: true	Kalau ada field asing yang dikirim, langsung error 400
transform: true	Tipe data otomatis dikonversi, misalnya string "1" jadi number 1

app.listen(3001)	Service ini jalan di port 3001
------------------	--------------------------------

2. api/services/kategori-service/src/app.module.ts

```
import { Module } from '@nestjs/common';
import { KategoriModule } from '../kategori/kategori.module';
import { PrismaModule } from '../prisma/prisma.module';

@Module({
  imports: [KategoriModule, PrismaModule],
  controllers: [],
  providers: [],
})
export class AppModule {}
```

File ini adalah module utama yang jadi pusat pendaftaran semua module yang ada di kategori-service.

Bagian	Fungsi
@Module	Decorator NestJS untuk mendefinisikan sebuah module
PrismaModule	Didaftarkan di sini supaya koneksi database tersedia di seluruh service
KategoriModule	Module yang berisi semua logic CRUD kategori
controllers: []	Kosong karena controller sudah dihandle di masing-masing module
providers: []	Kosong karena provider sudah dihandle di masing-masing module

3. api/services/kategori-service/src/prisma/prisma.module.ts

```
import { Global, Module } from '@nestjs/common';
import { PrismaService } from '../prisma.service';

@Global()
@Module({
  providers: [PrismaService],
  exports: [PrismaService],
})
export class PrismaModule {}
```

File ini adalah module khusus untuk Prisma yang bertanggung jawab menyediakan koneksi database ke seluruh aplikasi.

Bagian	Fungsi
@Global()	Menjadikan module ini global, jadi PrismaService bisa dipakai di module manapun tanpa perlu import ulang

@Module	Decorator NestJS untuk mendefinisikan sebuah module
providers: [PrismaService]	Mendaftarkan PrismaService supaya bisa di-inject ke service lain
exports: [PrismaService]	Mengekspos PrismaService supaya bisa dipakai di luar module ini

4. api/services/kategori-service/src/prisma/prisma.service.ts

```
import 'dotenv/config';
import { Injectable, OnModuleInit, OnModuleDestroy } from '@nestjs/common';
import { PrismaClient } from '../generated/prisma/client';
import { PrismaPg } from '@prisma/adapters-pg';

// Service untuk jadi jembatan antara NestJS dan Prisma
// Extends PrismaClient supaya bisa di-inject ke service lain
@Injectable()
export class PrismaService
  extends PrismaClient
  implements OnModuleInit, OnModuleDestroy
{
  constructor() {
    const adapter = new PrismaPg({
      connectionString: process.env['DATABASE_URL'] as string,
    });
    super({ adapter });
  }

  // Konek ke DB pas module pertama kali jalan
  async onModuleInit(): Promise<void> {
    await this.$connect();
  }

  // Putus koneksi DB pas module di-destroy biar gak memory leak
  async onModuleDestroy(): Promise<void> {
    await this.$disconnect();
  }
}
```

File ini adalah service utama yang mengatur koneksi ke database PostgreSQL menggunakan Prisma. Semua operasi database di kategori-service melewati service ini.

Bagian	Fungsi
import 'dotenv/config'	Memastikan variabel dari file .env terbaca sebelum apapun dijalankan
extends PrismaClient	PrismaService mewarisi semua method dari PrismaClient, jadi bisa langsung pakai this.prisma.kategori.findMany() dll
implements OnModuleInit, OnModuleDestroy	Mengikuti lifecycle NestJS — ada aksi saat module start dan saat module mati

PrismaPg	Adapter khusus untuk koneksi ke PostgreSQL lokal, wajib di Prisma 7
connectionString: process.env['DATABASE_URL']	URL koneksi database diambil dari .env
onModuleInit()	Dipanggil otomatis saat service pertama kali diinisialisasi — membuka koneksi ke database
onModuleDestroy()	Dipanggil otomatis saat aplikasi mati — menutup koneksi database supaya tidak terjadi memory leak

5. api/services/kategori-service/src/kategori/kategori.module.ts

```
import { Module } from '@nestjs/common';
import { KategoriService } from '../kategori.service';
import { KategoriController } from '../kategori.controller';

@Module({
  controllers: [KategoriController],
  providers: [KategoriService],
})
export class KategoriModule {}
```

File ini adalah module khusus kategori yang mengelompokkan controller dan service kategori menjadi satu kesatuan.

Bagian	Fungsi
@Module	Decorator NestJS untuk mendefinisikan sebuah module
controllers: [KategoriController]	Mendaftarkan KategoriController sebagai penerima request HTTP
providers: [KategoriService]	Mendaftarkan KategoriService sebagai logic handler yang bisa di-inject